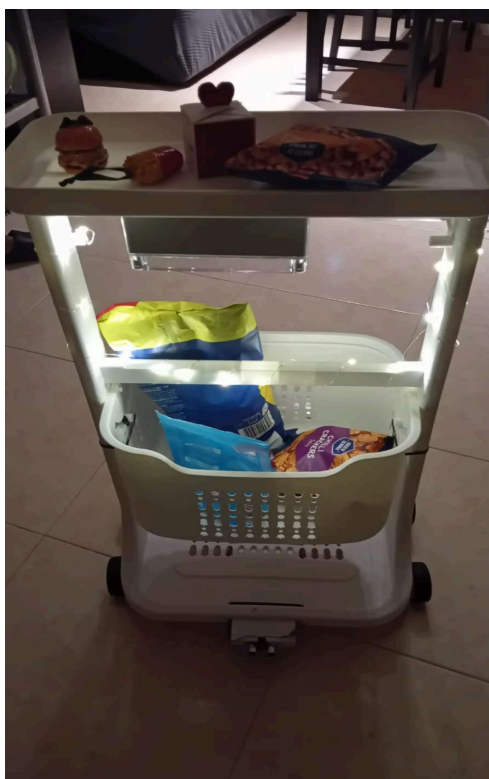


Projecto SnackTime

Relatório Final da Prova de Aptidão Profissional (PAP)



Discente: Hugo Luís Fernandes Robalo

Professor Orientador: Prof. Ricardo Pires

Diretor do Curso: Prof. Ricardo Pires

Diretor de Turma: Prof. Inês Silva

Sintra, Fevereiro 2026

Agradecimentos

Gostaria de expressar o meu profundo agradecimento a todos os que contribuíram para a realização deste projeto. Em especial, agradeço aos meus pais, que estiveram sempre ao meu lado, ajudando na aquisição dos componentes, na montagem do sistema e na construção das ideias que tornaram o SnackTime possível. O seu apoio constante, dedicação e incentivo foram fundamentais para que este projeto se concretizasse.

Agradeço igualmente aos professores que, ao longo do curso, contribuíram para a minha formação técnica e pessoal. O seu acompanhamento, disponibilidade e esclarecimento de dúvidas foram essenciais durante o desenvolvimento deste trabalho.

Por fim, deixo um agradecimento especial à minha mãe e ao meu pai pela força, motivação e palavras de carinho que me deram ao longo de todo o processo. Sem eles, este projeto não teria sido possível.

Resumo

O relatório descreve o desenvolvimento do projeto SnackTime, realizado no âmbito da PAP do Curso Profissional de Técnico de Gestão de Equipamentos Informáticos. O projeto consiste numa mesa telecomandada, controlada remotamente através de Bluetooth, utilizando um microcontrolador Arduino UNO R4 WIFI, um módulo driver controlador de motores L298N, motores DC e um sensor de obstáculos ultrassônico. O principal objetivo do projeto é permitir o controlo remoto da mesa de forma simples, eficiente e segura, demonstrando a aplicação prática de conhecimentos adquiridos nas áreas da eletrónica, programação, robótica e sistemas de comunicação.

Índice

1.Introdução.....	Pág.5
2.Diagrama de Gantt.....	Pág.6
3.Enquadramento Teórico.....	Pág.7
4.Descrição e Fundamentação da Escolha do Projeto.....	Pág.8
5. Desenvolvimento do Projeto.....	Pág.9
5.1 Arquitectura do Sistema.....	Pág.9
5.1.1 Unidade de Control.....	Pág.9
5.1.2 Comunicação.....	Pág.9
5.1.3 Sensores	Pág.10
5.1.4 Sistema de Atuação.....	Pág.11
5.1.5 Interface Visual.....	Pág.11
5.1.6 Sistema de Alimentação.....	Pág.12
5.2 Diagrama de Blocos/Componentes Utilizados.....	Pág.13
5.2.1 Imagens dos Componentes utilizados.....	Pág.14
5.3 Máquina de Estados.....	Pág.16
5.4 Código da Comunicação Bluetooth.....	Pág.17
5.5 Código do Controlo de Motores e Sensor de Obstáculos.....	Pág.18
6. Opções Estratégicas.....	Pág.22
7. Reflexão Crítica da Execução do Projeto.....	Pág.23
8. Conclusão.....	Pág.24
9. Webgrafia.....	Pág.25
10. Anexos.....	Pág.26

Lista de Abreviaturas/Siglas

- PAP (Prova de Aptidão Profissional);
- DC (Direct Current/Corrente Contínua);
- BLE (Bluetooth Low Energy);

1. Introdução

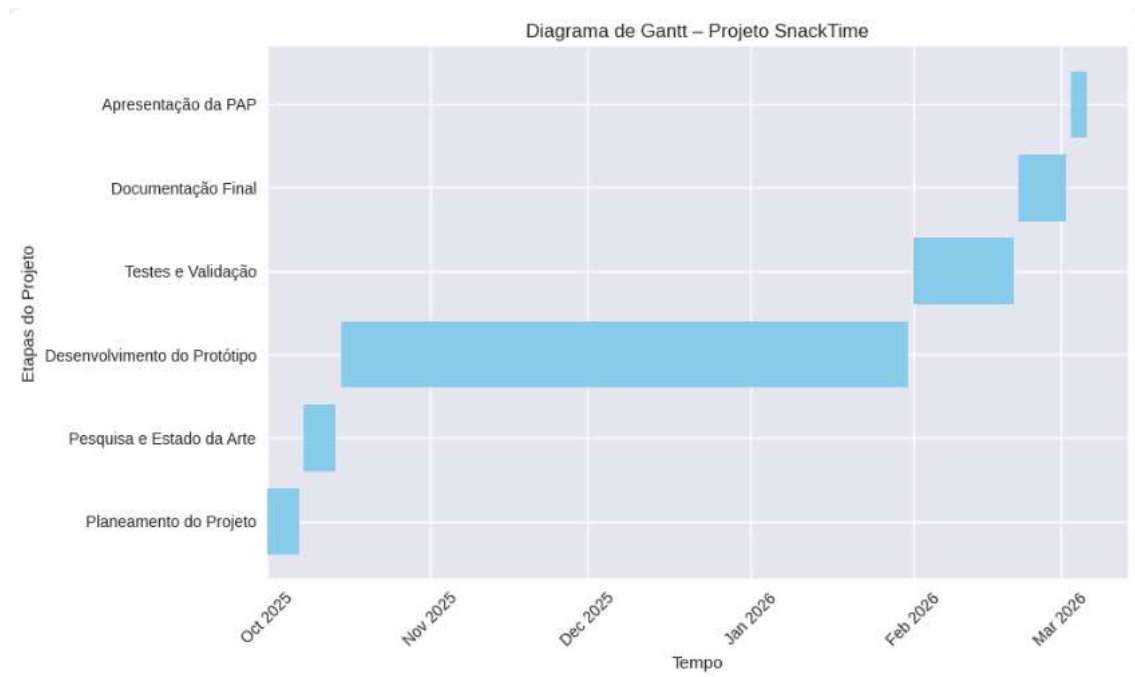
Este projeto foi desenvolvido no âmbito da Prova de Aptidão Profissional (PAP) do Curso Profissional de Técnico de Gestão de Equipamentos Informáticos. O SnackTime consiste numa mesa telecomandada destinada a apoiar pessoas com mobilidade reduzida, permitindo que esta se desloque remotamente através de comandos enviados por uma aplicação móvel via Bluetooth.

A programação e a montagem do sistema possibilitaram a aplicação prática dos conhecimentos adquiridos ao longo do curso, nomeadamente nas disciplinas de Sistemas Digitais e Arquitetura de Computadores, Eletrónica, Comunicação de Dados e Programação. Este projeto representou, assim, uma oportunidade para integrar diferentes áreas tecnológicas num único sistema funcional.

O principal objetivo deste trabalho é demonstrar a capacidade de conceber, desenvolver e implementar uma solução tecnológica fiável, prática e com utilidade real no dia a dia, evidenciando a integração eficaz entre hardware e software.

2. Diagrama de Gantt

O desenvolvimento do projeto foi distribuído ao longo do ano letivo, passando pelos seis meses e pelas seguintes fases:



O que mostra no Diagrama de Gantt:

- Pesquisa e planeamento do projeto
- Desenvolvimento do Protótipo:
 - Seleção dos componentes eletrónicos;
 - Montagem do circuito e da estrutura física;
 - Programação do Arduino;
- Testes de Validação e funcionamento:
 - Correções e otimizações;
 - Elaboração do relatório final.
- Documentação Final
- Apresentação da PAP

3. Enquadramento Teórico

Este projeto de final de ano enquadra-se na área da robótica e dos sistemas de controlo remoto, que têm vindo a assumir um papel cada vez mais relevante na sociedade moderna, sendo amplamente utilizados na indústria, logística, saúde e domótica. Um exemplo disso é o restaurante Go Chef, em Massamá, que utiliza robôs para servir às mesas.

Para o desenvolvimento deste projeto recorri a microcontroladores, como o Arduino, que permitem criar sistemas de baixo custo, flexíveis e facilmente programáveis. A comunicação Bluetooth foi utilizada como tecnologia sem fios para o controlo remoto do dispositivo, devido à sua simplicidade, baixo consumo energético e compatibilidade com smartphones.

Os motores DC, quando controlados através do módulo L298N, permitem ajustar o sentido e a velocidade de rotação, tornando-os ideais para projetos de mobilidade.

4. Descrição e Fundamentação da Escolha do Projeto

A escolha deste projeto deveu-se às ideias criativas que eu e o meu pai fomos tendo, com especial foco nas necessidades das pessoas com mobilidade reduzida, bem como, no meu interesse pessoal pela área da robótica e do hardware. Além disso, surgiu a vontade de criar algo fora da caixa, recorrendo a um sistema funcional que integrasse eletrônica, programação e comunicação sem fios.

O projeto SnackTime permite aplicar, de forma prática, vários conhecimentos adquiridos ao longo do curso e, ao mesmo tempo, representa um desafio técnico interessante e adequado ao nível de ensino.

Ao escolher este projeto, pensei no quão divertido poderia ser desenvolvê-lo, mesmo sabendo que seria complicado encontrar a forma ideal de o implementar. Ainda assim, considerei a ideia suficientemente apelativa para avançar. No futuro, pretendo melhorar o sistema e, quem sabe, transformá-lo num produto acessível, de baixo custo e com potencial para ser comercializado.

5. Desenvolvimento do Projeto

5.1 Arquitetura do Sistema

O sistema desenvolvido é composto por seis blocos principais: controlo, comunicação, sensores, atuação, interface visual e alimentação.

5.1.1 Unidade de Controlo – Arduino UNO R4 WiFi

O **Arduino UNO R4 WiFi** é o elemento central do sistema, responsável por:

- Processamento de todos os dados;
- Gestão da máquina de estados (Avançar, Recuar, Esquerda, Direita, Parar);
- Leitura dos sensores ultrassónicos;
- Controlo do driver de motores;
- Comunicação Bluetooth Low Energy (BLE);
- Gestão da matriz LED integrada.

A programação foi desenvolvida em linguagem C/C++ utilizando o ambiente Arduino IDE.

5.1.2 Comunicação – Bluetooth Low Energy (BLE)

A comunicação entre o robô e o utilizador é realizada através do **BLE** integrado no **Arduino UNO R4 WiFi**, utilizando a biblioteca ArduinoBLE.

Características da Comunicação:

- Nome do dispositivo: Arduino do Hugão
- Tipo de comunicação: BLE (Byte Characteristic)
- Alcance aproximado: até 10 metros

Comandos Implementados:

- **'F'** - Avançar
- **'B'** - Recuar
- **'L'** - Virar à esquerda
- **'R'** - Virar à direita
- **Outro valor** - Parar

Não são utilizados pinos físicos TX/RX externos, pois o módulo BLE é interno à placa.

5.1.3. Sensores – Ultrasónicos HC-SR04

O sistema utiliza dois sensores ultrassónicos HC-SR04 para deteção de obstáculos:

- Um sensor na parte frontal;
- Um sensor na parte traseira.

Ligações dos Sensores

Sensor Frontal:

- Trigger → Pino D2
- Echo → Pino D3

Sensor Traseiro:

- Trigger → Pino D4
- Echo → Pino D7

Funcionamento:

- O Arduino envia um pulso de 10 μ s no pino Trigger;
- Mede o tempo de retorno no pino Echo;
- Calcula a distância com base na velocidade do som;
- Caso a distância seja inferior ou igual a 20 cm, o movimento nessa direção é bloqueado.

5.1.4 Sistema de Atuação – Driver L298N e Motores DC

O controlo dos motores é realizado através do módulo driver **L298N**, que permite controlar dois motores DC de forma independente.

Ligações ao Arduino:

IN1 - Pino 8

IN2 - Pino 9

IN3 - Pino 10

IN4 - Pino 11

ENA(PWM Motor A) - Pino 5

ENB(PWM Motor B) - Pino 6

Funcionamento:

- IN1 até IN4 determinam o sentido de rotação dos motores;
- ENA e ENB permitem controlo de velocidade por PWM;
- A velocidade foi definida no código com valor 250 (escala 0–255).

O sistema permite:

- Movimento para a frente;
- Movimento para trás;
- Rotação à esquerda;
- Rotação à direita;
- Paragem total.

5.1.5. Interface Visual – Matriz LED Integrada

O Arduino UNO R4 WiFi possui uma matriz LED integrada que foi utilizada para:

- Exibir a mensagem/animação “NO!” quando é detetado obstáculo frontal;
- Fornecer feedback visual ao utilizador.

Esta matriz não requer pinos adicionais, pois está integrada na placa.

5.1.6. Sistema de Alimentação – Pilhas 18650

O sistema é alimentado por **duas pilhas recarregáveis 18650**, ligadas em série.

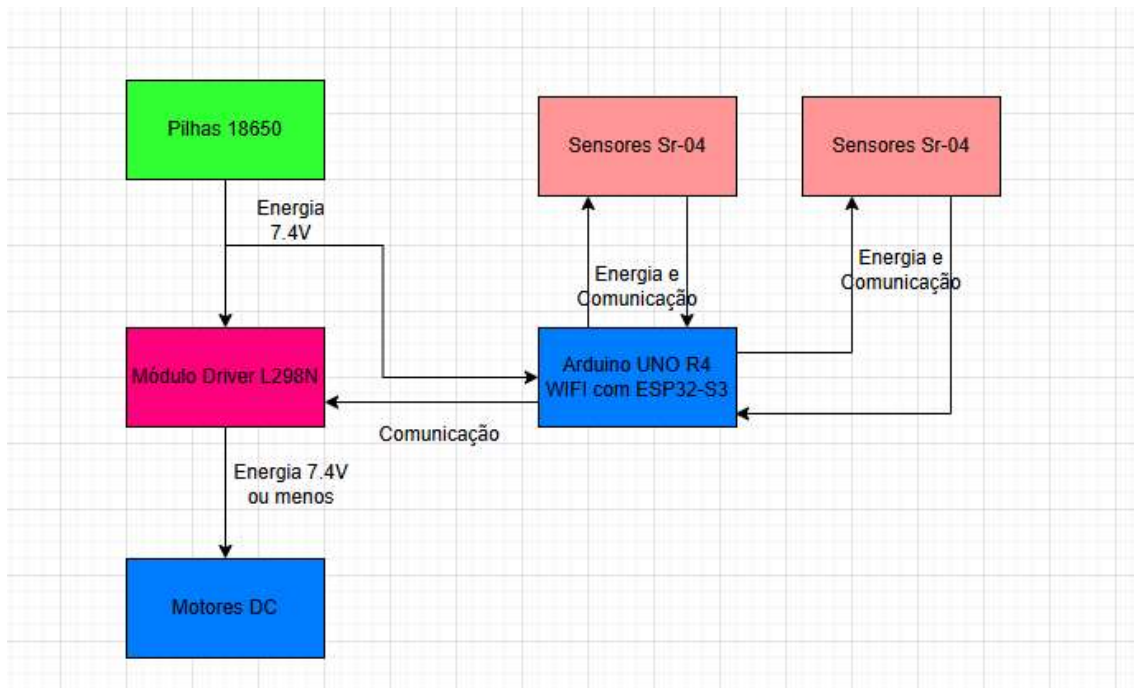
Características:

- Tensão nominal: 3.7 V cada
- Tensão total do sistema: aproximadamente 7.4 V
- Elevada capacidade energética
- Boa autonomia para sistemas móveis

A alimentação fornece energia:

- Ao Arduino UNO R4 WiFi;
Ao módulo driver L298N;
- Aos motores DC;
- Aos sensores ultrassónicos.

5.2 Diagrama de Blocos(Simples)/Componentes Utilizados



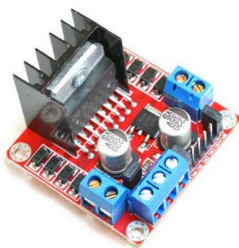
As baterias **18650 de lítio** constituem a principal fonte de alimentação do sistema. A energia fornecida por estas baterias é encaminhada para o **driver de motor L298N**, que a distribui para os motores DC responsáveis pelo movimento da mesa. O **Arduino UNO R4 WiFi** é igualmente alimentado pelas mesmas baterias e, enquanto os sensores são alimentados diretamente através do próprio Arduino. Esta configuração garante uma distribuição de energia estável e adequada ao funcionamento de todos os componentes do sistema.

5.2.1 Imagem dos Componentes Utilizados

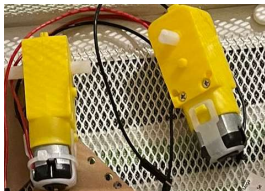
Arduino UNO R4 WiFi;



Módulo driver controlador de motores L298N;



Motores DC;



Sensor ultrassônico SR-04;



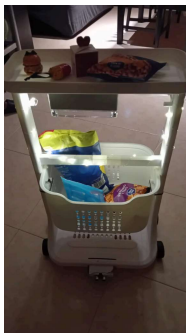
Pilhas recarregáveis 18650;



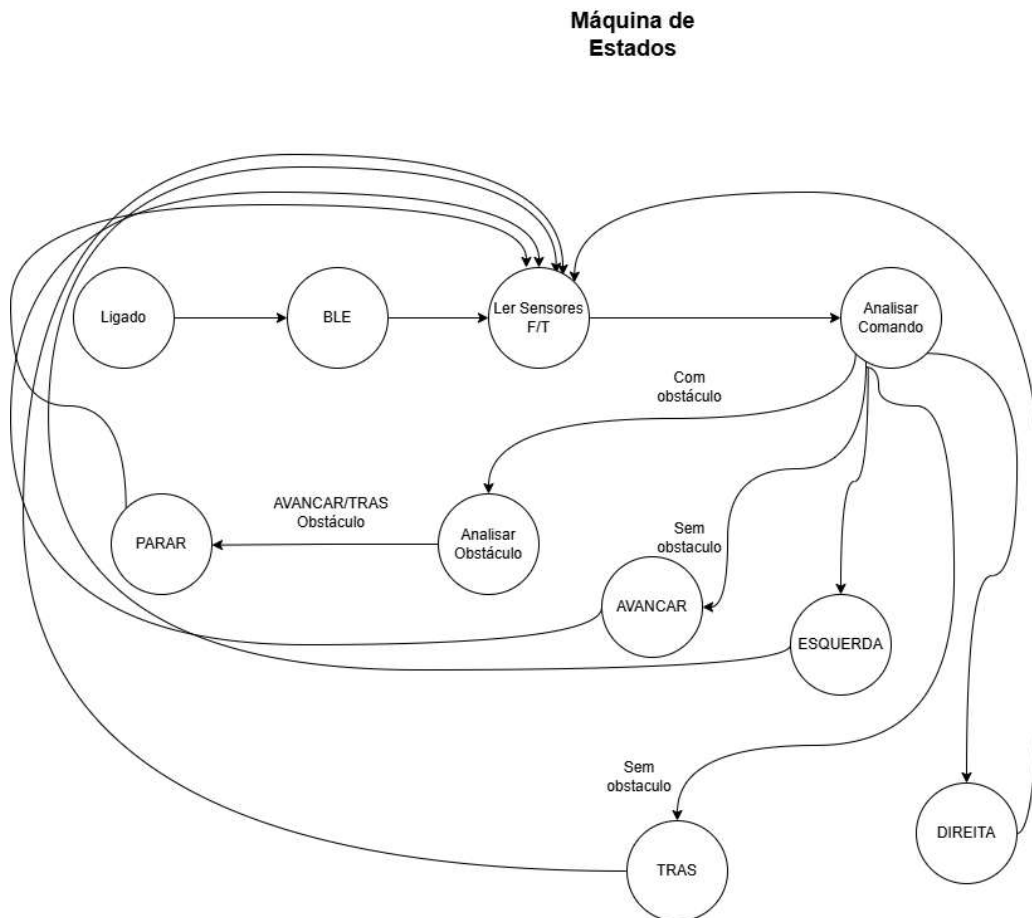
Cabos de ligação;



Estrutura da mesa;



5.3 Máquina de Estados



Quando o projeto **SnackTime** é ligado, o sistema liga o modo BLE(Bluetooth Low Energy), de seguida irá Ler os sensores da Frente e de Trás, aguardando que o utilizador selecione um comando através da aplicação móvel. Assim que o utilizador envia uma instrução, como avançar ou recuar, o microcontrolador ativa os sensores ultrassónicos para analisar obstáculos, esses sensores colocados na parte frontal e traseira da mesa.

Durante o movimento, estes sensores monitoram continuamente a distância entre a mesa e eventuais obstáculos. Caso a mesa esteja a deslocar-se para a frente e o sensor frontal detete um objeto a menos de 20 cm, o sistema interrompe imediatamente o movimento. A ordem de avançar fica temporariamente bloqueada até que o obstáculo deixe de ser detetado ou até que o utilizador envie um novo comando. O mesmo comportamento ocorre quando a mesa se encontra a recuar e o sensor traseiro identifica um obstáculo dentro da mesma distância de segurança.

Este mecanismo garante não só a segurança do utilizador e do equipamento, como também aumenta a fiabilidade do sistema, evitando colisões e permitindo um controlo mais preciso e seguro da mesa telecomandada.

5.4 Código de Comunicação Bluetooth

A comunicação entre o telemóvel e o sistema é realizada através de Bluetooth Low Energy (BLE). Os comandos enviados pela aplicação permitem controlar os movimentos da mesa em tempo real.

No Código:

- Incluímos a biblioteca “ArduinoBLE.h”

```
#include <ArduinoBLE.h>
```

Na imagem abaixo, o código cria um serviço BLE chamado robotService com um UUID específico e, de seguida, define uma característica chamada commandChar associada a esse serviço. A característica permite leitura e escrita (BLERead | BLEWrite), o que significa que um dispositivo externo, como um telemóvel, pode tanto enviar comandos como ler informações dessa característica.

```
BLEService robotService("18810001-E8F2-537E-4F6C-D104768A1215");
BLEByteCharacteristic commandChar(
  "18810002-E8F2-537E-4F6C-D104768A1215",
  BLERead | BLEWrite
```

-Na imagem abaixo o código inicia o módulo BLE, define o nome do dispositivo como "Arduino do Hugão" e associa o serviço e a característica criada atrás. De seguida, adiciona o serviço ao BLE e começa a anunciar para que outros dispositivos, como um telemóvel, para que possa se conectar.

```
BLE.begin();
BLE.setLocalName("Arduino do Hugão");
BLE.setAdvertisedService(robotService);
robotService.addCharacteristic(commandChar);
BLE.addService(robotService);
BLE.advertise();
```

- Na imagem abaixo verifica se algum telemóvel já se conectou ou não

```
BLEDevice central = BLE.central();
```

Na imagem abaixo, o código verifica se existe um dispositivo central ligado, e quando estiver conectado, mantém-se em execução. Sempre que recebe um novo valor na característica commandChar, lê esse valor e chama a função atualizarEstado() para processar a nova instrução.

```
if (central) {  
  
    while (central.connected()) {  
  
        if (commandChar.written()) {  
            command = commandChar.value();  
            atualizarEstado();  
        }  
    }  
}
```

5.5 Controlo de Motores e Sensor de Obstáculos

Nesta secção descreve-se o modo como o sistema controla os motores DC através do driver L298N e como utiliza dois sensores ultrassónicos HC-SR04 um na parte frontal e outro na traseira — para detetar obstáculos e impedir movimentos potencialmente perigosos. O controlo do movimento é realizado por Bluetooth (BLE), permitindo que o utilizador envie comandos a partir de uma aplicação móvel. Toda a lógica de funcionamento é estruturada com base numa máquina de estados, garantindo um comportamento organizado, previsível e seguro durante a operação do dispositivo.

Definição dos pinos dos motores (L298N):

Explicação:

Estes pinos controlam o driver L298N.

IN1 até IN4 definem o sentido de rotação.

ENA e ENB controlam a velocidade por PWM.

A variável velocidade define a potência aplicada aos motores.

```
// Pinos do Motor L298N
const byte IN1 = 8;
const byte IN2 = 9;
const byte IN3 = 10;
const byte IN4 = 11;
const byte ENA = 5;
const byte ENB = 6;

byte velocidade = 250;
```

Definição dos sensores ultrassônicos:

Explicação:

Define os pinos TRIG e ECHO de cada sensor.

Um sensor está na frente e outro atrás.

Permitem medir distância a obstáculos.

```
// Sensor da Frente e Trás
const int trigPinFront = 2;
const int echoPinFront = 3;
const int trigPinBack = 4;
const int echoPinBack = 7;
```

Função de leitura da distância:

Explicação:

Envia pulso ultrassônico.

Mede o tempo de retorno.

Converte tempo em distância (cm).

Retorna 999 se não houver leitura válida.

```
float lerDistancia(int trigPin, int echoPin) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    duration = pulseIn(echoPin, HIGH, 30000);
    if (duration == 0) return 999;
    return duration * 0.034 / 2;
}
```

Bloqueio de obstáculos:

Explicação:

Se a distância for ≤ 20 cm, ativa o bloqueio.

Impede movimento nessa direção.

```
bloqueioFrente = (distFrente <= 20);
bloqueioTras   = (distTras   <= 20);
```

Máquina de Estados Implementada:

Explicação:

Sistema organizado por estados.

Apenas um movimento ativo de cada vez.

Estrutura organizada e segura.

```
// Estados da maquina
char command = 'S';

#define AVANCAR 1
#define TRAS 2
#define DIREITA 3
#define ESQUERDA 4
#define PARAR 5

int estadoAtual = PARAR;
```

```
// Máquina de estados
switch (estadoAtual) {
    case AVANCAR:
        moveForward();
        break;

    case TRAS:
        moveBackward();
        break;

    case ESQUERDA:
        turnLeft();
        break;

    case DIREITA:
        turnRight();
        break;

    case PARAR:
    default:
        stopAll();
        break;
}
```

Funções de movimento:

Isto mostra como o L298N é controlado e onde ele dá energia às rodas.

```
// Movimentos
void moveForward() {
  analogWrite(ENA, velocidade);
  analogWrite(ENB, velocidade);
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}

void moveBackward() {
  analogWrite(ENA, velocidade);
  analogWrite(ENB, velocidade);
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}

void turnLeft() {
  analogWrite(ENA, velocidade);
  analogWrite(ENB, velocidade);
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}

void turnRight() {
  analogWrite(ENA, velocidade);
  analogWrite(ENB, velocidade);
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}

void stopAll() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}
```

Interceção de movimento perigoso:

E também o equivalente para trás. A imagem mostra que o sistema não depende apenas do comando ,ele verifica continuamente os sensores

```
if (estadoAtual == AVANCAR && bloqueioFrente) {
  estadoAtual = PARAR;
```

```
if (estadoAtual == TRAS && bloqueioTras) {  
    estadoAtual = PARAR;
```

6. Opções Estratégicas

As opções estratégicas tomadas ao longo do desenvolvimento do projeto SnackTime tiveram como objetivo garantir simplicidade, fiabilidade e baixo custo, sem comprometer a funcionalidade. A escolha do **Arduino UNO R4 WiFi** deveu-se ao facto de integrar **Bluetooth Low Energy (BLE)**, permitindo uma comunicação direta com smartphones sem necessidade de módulos adicionais. Esta característica reduziu a complexidade do sistema e facilitou o controlo remoto da mesa.

A seleção dos **sensores ultrassónicos HC-SR04** foi estratégica devido ao seu **baixo custo**, **boa precisão** e facilidade de integração, assegurando uma deteção eficaz de obstáculos tanto à frente como atrás da mesa.

O **driver L298N** foi escolhido por suportar a corrente necessária para os motores DC e por ser uma solução económica e amplamente utilizada em projetos educativos, garantindo fiabilidade no controlo do movimento.

A utilização da aplicação **BLE Controller**, disponível na Play Store, permitiu implementar rapidamente o controlo remoto sem necessidade de desenvolver uma aplicação própria, reduzindo tempo e complexidade.

Por fim, a opção por **baterias de lítio recarregáveis** garantiu uma excelente autonomia e portabilidade, tornando o sistema mais prático e adequado ao uso diário.

7. Reflexão Crítica da Execução do Projeto

Durante a realização deste projeto surgiram diversas dificuldades, sobretudo ao nível da programação e da integração entre os diferentes componentes. No entanto, cada obstáculo contribuiu para o desenvolvimento de novas competências, para o reforço da autonomia e para a melhoria da capacidade de resolver problemas de forma prática e eficiente.

Apesar das limitações de tempo e de recursos, considero que os objetivos principais foram alcançados, resultando num sistema funcional, estável e com potencial real de evolução. Este projeto não só permitiu consolidar conhecimentos adquiridos ao longo do curso, como também abriu portas para melhorias futuras que poderão tornar o SnackTime ainda mais útil e inovador.

Acredito que este sistema pode vir a ser bastante proveitoso no futuro, especialmente para pessoas com mobilidade reduzida, oferecendo-lhes maior autonomia e conforto no dia a dia. Além disso, trata-se de um projeto que, para além de funcional, pode ser divertido de utilizar em ambiente doméstico. Uma das melhorias que pretendo implementar no futuro é a capacidade de a mesa se deslocar autonomamente pela casa e vir até ao utilizador, tornando o sistema ainda mais inteligente e prático.

8. Conclusão

O projeto **SnackTime** permitiu consolidar os conhecimentos técnicos adquiridos ao longo do curso e desenvolver competências práticas nas áreas de eletrónica, programação, robótica e integração de sistemas. Ao longo do processo, surgiram vários desafios, sobretudo na programação e na articulação entre os diferentes componentes, mas cada dificuldade contribuiu para o crescimento pessoal, para o reforço da autonomia e para a capacidade de resolver problemas de forma eficaz.

O sistema desenvolvido demonstra a capacidade de planear, construir e implementar uma solução tecnológica funcional, cumprindo os objetivos definidos para a Prova de Aptidão Profissional. Para além disso, revelou-se um projeto com potencial real de evolução e aplicação prática, especialmente no apoio a pessoas com mobilidade reduzida, podendo no futuro tornar-se uma ferramenta útil no seu dia a dia.

Apesar das limitações de tempo e recursos, o resultado final é um protótipo estável, seguro e funcional. Acredita-se que, com melhorias futuras — como a possibilidade de a mesa se deslocar autonomamente pela casa e dirigir-se até ao utilizador, o SnackTime poderá transformar-se num produto acessível, inovador e até divertido de utilizar em ambiente doméstico.

Este projeto representa, assim, não apenas a conclusão de um percurso formativo, mas também o início de novas oportunidades de aprendizagem e desenvolvimento tecnológico.

10. Webgrafia

<https://store-usa.arduino.cc/products/uno-r4-wifi>

<https://jsanders.myds.me/Help/Arduino/Reference/www.arduino.cc/en/Reference/BLEServiceConstructor.html>

<https://www.youtube.com/watch?v=vro4zqYeLWc>

<https://forum.arduino.cc/t/help-with-the-arduino-ble/1103012>

<https://youtu.be/aWScX8IOEfw?si=jUDGMBCMqb4ZKEK1>

<https://youtu.be/gU-CZP2nIwQ?si=vza6VSRYrhaKayPP>

<https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>

https://www.mouser.com/datasheet/3/1524/1/ASR00050_18650_2500mAh.pdf?srltid=AfmBOore2hy5MeLS0ws3VmmzD-KTrqWODcQqBggcJlxWwexVgLHBZW

[https://www.raveo.cz/sites/default/files/dkm/katalogy/motory/DC%20MOTOR%20\(15W~120W\).pdf](https://www.raveo.cz/sites/default/files/dkm/katalogy/motory/DC%20MOTOR%20(15W~120W).pdf)

<https://www.handsontec.com/dataspecs/L298N%20Motor%20Driver.pdf>

https://documentation.espressif.com/esp32-s3_datasheet_en.pdf

<https://mauser.pt/>

<https://projecthub.arduino.cc/>

<https://www.instructables.com/Arduino-Bluetooth-Car-Controlled-by-Smartphone-Eas/>

11. Anexos

-Código fonte do Arduino e a sua explicação:

Este programa controla um carro telecomandado por Bluetooth (BLE), utilizando um **Arduino UNO R4 WiFi**, dois sensores ultrassónicos (frontal e traseiro) para evitar colisões e a matriz LED integrada, que exibe a animação “NO!” sempre que é detetado um obstáculo à frente. No início, são incluídas as bibliotecas necessárias: **ArduinoBLE** para comunicação Bluetooth Low Energy, **Arduino_LED_Matrix** para controlar a matriz LED e o ficheiro **animation.h**, que contém a animação apresentada quando ocorre um bloqueio frontal.

É criado o objeto *matrix* para gerir a matriz LED e são definidos os pinos do driver **L298N**: os pinos IN1–IN4 (D8, D9, D10, D11) determinam o sentido de rotação dos motores, enquanto os pinos ENA (D5) e ENB (D6) utilizam PWM para controlar a velocidade. A velocidade base é definida na variável *velocidade*, com o valor 250 (numa escala de 0 a 255).

Seguidamente, são configurados os pinos dos sensores ultrassónicos: na frente, Trigger em D2 e Echo em D3; atrás, Trigger em D4 e Echo em D7. O programa utiliza variáveis para armazenar a duração do eco (*duration*) e as distâncias calculadas (*distFrente* e *distTras*). Existem também variáveis booleanas de bloqueio (*bloqueioFrente* e *bloqueioTras*), que indicam a presença de obstáculos, e a variável *jaMostrouFrente*, que impede que a animação seja repetida continuamente enquanto o obstáculo se mantém.

Na parte do Bluetooth, o código cria um serviço BLE (*robotService*) e uma característica (*commandChar*) do tipo byte, suficiente para enviar comandos simples como ‘F’, ‘B’, ‘L’ ou ‘R’. A variável *command* guarda o comando atual e inicia com ‘S’, que funciona como “stop”, garantindo que o carro permanece

parado ao arrancar. A máquina de estados é representada por constantes (AVANCAR, TRAS, DIREITA, ESQUERDA, PARAR), e a variável *estadoAtual* começa em PARAR.

No *setup()*, o Arduino inicializa a comunicação serial, configura os pinos do L298N como saídas, define os pinos Trigger e Echo dos sensores, inicia e limpa a matriz LED. Em seguida, inicializa o BLE, define o nome do dispositivo (“Arduino do Hugão”), anuncia o serviço, adiciona a característica e começa a transmitir para permitir a ligação de um telemóvel.

No *loop()*, o Arduino aguarda que um dispositivo BLE se ligue. Quando a ligação é estabelecida, entra num ciclo enquanto o telemóvel permanecer conectado. Nesse ciclo, o Arduino mede continuamente as distâncias à frente e atrás através da função *lerDistancia()*. Esta função envia um pulso no Trigger e mede o tempo em que o Echo permanece HIGH usando *pulseIn*. Se não houver retorno, devolve 999 cm; caso contrário, calcula a distância multiplicando o tempo por 0.034 (velocidade aproximada do som em cm/μs) e dividindo por 2.

Com as distâncias calculadas, o programa atualiza os bloqueios: se a distância for igual ou inferior a 20 cm, ativa o bloqueio correspondente. Se a característica BLE tiver sido escrita, o Arduino lê o comando recebido e chama *atualizarEstado()*, que converte o comando num estado da máquina, garantindo segurança: ‘F’ só avança se não houver bloqueio frontal; ‘B’ só recua se não houver bloqueio traseiro; ‘L’ e ‘R’ são aceites diretamente; qualquer outro comando resulta em PARAR.

Mesmo após definir o estado, o programa realiza uma verificação adicional de segurança. Se o estado for AVANCAR e surgir um obstáculo, chama *stopAll()* e mostra a animação “NO!” apenas uma vez, graças à variável *jaMostrouFrente*. Quando o obstáculo desaparece, essa variável é reposta a *false*. Para TRAS, o comportamento é semelhante, mas sem animação.

Se não existirem bloqueios, o programa executa o movimento correspondente através de um *switch*: *moveForward()*, *moveBackward()*, *turnLeft()*, *turnRight()* ou *stopAll()*.

As funções de movimento controlam diretamente o L298N: *moveForward()* aplica PWM nos pinos ENA e ENB e configura os INs para avançar; *moveBackward()* inverte os INs; *turnLeft()* e *turnRight()* fazem os motores rodar em sentidos opostos; *stopAll()* desliga os INs e coloca o PWM a zero. Um *delay(30)* no ciclo principal garante estabilidade nas leituras e no controlo.

Quando o telemóvel se desliga, o programa sai do ciclo, pára o carro e limpa a matriz LED, ficando pronto para uma nova ligação BLE.

```

1  /*
2  Programador.....: (C) Hugo Fernandes
3  Data.....: 29/12/2025
4  Observações.....: Arduino UNO R4 WiFi, Carro Bluetooth, switch case da Máquina de Estados
5  HC-SR04 Frente e na parte de Trás.
6  Matriz LED a dizer "NO!" quando encontra obstáculo na parte da frente ou na parte de trás.
7  */
8
9  #include <ArduinoBLE.h>
10 #include <Arduino_LED_Matrix.h>
11 #include "animation.h"
12
13 ArduinoLEDMatrix matrix;
14
15 // Pinos do Motor L298N
16 const byte IN1 = 8;
17 const byte IN2 = 9;
18 const byte IN3 = 10;
19 const byte IN4 = 11;
20 const byte ENA = 5;
21 const byte ENB = 6;
22
23 byte velocidade = 250;
24
25 // Sensor da Frente e Trás
26 const int trigPinFront = 2;
27 const int echoPinFront = 3;
28 const int trigPinBack = 4;
29 const int echoPinBack = 7;
30
31 long duration;
32 float distFrente;
33 float distTras;
34
35 // Bloqueios
36 bool bloqueioFrente = false;
37 bool bloqueioTras = false;
38
39 // Para não repetir animação sem parar
40 bool jaMostrouFrente = false;
41
42 // Bluetooth
43 BLEService robotService("18B10001-E8F2-537E-4F6C-D104768A1215");
44 BLEByteCharacteristic commandChar(
45     "18B10002-E8F2-537E-4F6C-D104768A1215",
46     BLERead | BLEWrite
47 );
48
49 // Estados da máquina
50 char command = 'S';

```

```

52 #define AVANCAR 1
53 #define TRAS 2
54 #define DIREITA 3
55 #define ESQUERDA 4
56 #define PARAR 5
57
58 int estadoAtual = PARAR;
59
60 // Setup
61 void setup() {
62     Serial.begin(9600);
63
64     pinMode(IN1, OUTPUT);
65     pinMode(IN2, OUTPUT);
66     pinMode(IN3, OUTPUT);
67     pinMode(IN4, OUTPUT);
68     pinMode(ENA, OUTPUT);
69     pinMode(ENB, OUTPUT);
70
71     pinMode(trigPinFront, OUTPUT);
72     pinMode(echoPinFront, INPUT);
73
74     pinMode(trigPinBack, OUTPUT);
75     pinMode(echoPinBack, INPUT);
76
77     matrix.begin();
78     matrix.clear();
79
80     BLE.begin();
81     BLE.setLocalName("Arduino do Hugão");
82     BLE.setAdvertisedService(robotService);
83     robotService.addCharacteristic(commandChar);
84     BLE.addService(robotService);
85     BLE.advertise();
86 }
87
88 // Loop
89 void loop() {
90     BLEDevice central = BLE.central();
91
92     if (central) {
93         while (central.connected()) {
94
95             // Ler sensores
96             distFrente = lerDistancia(trigPinFront, echoPinFront);
97             distTras = lerDistancia(trigPinBack, echoPinBack);
98
99             // Atualiza bloqueios
100             bloqueioFrente = (distFrente <= 20);

```

```

100     bloqueioFrente = (distFrente <= 20);
101     bloqueioTras   = (distTras   <= 20);
102
103     // Se recebeu comando
104     if (commandChar.written()) {
105         command = commandChar.value();
106         atualizarEstado(); // aqui já respeita bloqueios
107     }
108
109     // Intercepta movimento perigoso
110     // Frente bloqueada e está a tentar avançar -> pára e mostra animação (1x por ev
111     if (estadoAtual == AVANCAR && bloqueioFrente) {
112         estadoAtual = PARAR;
113
114         if (!jaMostrouFrente) {
115             showObstacleAnimation();
116             jaMostrouFrente = true;
117         }
118     } else {
119         // se já não há obstáculo à frente, volta a permitir mostrar numa próxima vez
120         if (!bloqueioFrente) jaMostrouFrente = false;
121
122         // Trás bloqueada e está a tentar recuar -> pára (sem animação)
123         if (estadoAtual == TRAS && bloqueioTras) {
124             estadoAtual = PARAR;
125         } else {
126             // Máquina de estados
127             switch (estadoAtual) {
128                 case AVANCAR:
129                     moveForward();
130                     break;
131

```

```
131
132         case TRAS:
133             moveBackward();
134             break;
135
136         case ESQUERDA:
137             turnLeft();
138             break;
139
140         case DIREITA:
141             turnRight();
142             break;
143
144         case PARAR:
145         default:
146             stopAll();
147             break;
148     }
149 }
150 }
151
152 delay(30);
153 }
154
```

```

147         break;
148     }
149 }
150 }
151
152     delay(30);
153 }
154
155     stopAll();
156     matrix.clear();
157 }
158 }
159
160 // Funções e Estado
161
162 void atualizarEstado() {
163     // Bloqueia só o comando perigoso
164     if (command == 'F') {
165         if (!bloqueioFrente) estadoAtual = AVANCAR;
166         else estadoAtual = PARAR; // ignora F (fica parado)
167     }
168     else if (command == 'B') {
169         if (!bloqueioTras) estadoAtual = TRAS;
170         else estadoAtual = PARAR; // ignora B (fica parado)
171     }
172     else if (command == 'L') estadoAtual = ESQUERDA;
173     else if (command == 'R') estadoAtual = DIREITA;
174     else estadoAtual = PARAR;
175 }
176
177 float lerDistancia(int trigPin, int echoPin) {
178     digitalWrite(trigPin, LOW);
179     delayMicroseconds(2);
180     digitalWrite(trigPin, HIGH);
181     delayMicroseconds(10);
182     digitalWrite(trigPin, LOW);
183
184     duration = pulseIn(echoPin, HIGH, 30000);
185     if (duration == 0) return 999;
186     return duration * 0.034 / 2;
187 }
188
189 // Movimentos
190 void moveForward() {
191     analogWrite(ENA, velocidade);
192     analogWrite(ENB, velocidade);
193     digitalWrite(IN1, LOW);
194     digitalWrite(IN2, HIGH);
195     digitalWrite(IN3, LOW);

```

```

188
189 // Movimentos
190 void moveForward() {
191     analogWrite(ENA, velocidade);
192     analogWrite(ENB, velocidade);
193     digitalWrite(IN1, LOW);
194     digitalWrite(IN2, HIGH);
195     digitalWrite(IN3, LOW);
196     digitalWrite(IN4, HIGH);
197 }
198
199 void moveBackward() {
200     analogWrite(ENA, velocidade);
201     analogWrite(ENB, velocidade);
202     digitalWrite(IN1, HIGH);
203     digitalWrite(IN2, LOW);
204     digitalWrite(IN3, HIGH);
205     digitalWrite(IN4, LOW);
206 }
207
208 void turnLeft() {
209     analogWrite(ENA, velocidade);
210     analogWrite(ENB, velocidade);
211     digitalWrite(IN1, HIGH);
212     digitalWrite(IN2, LOW);
213     digitalWrite(IN3, LOW);
214     digitalWrite(IN4, HIGH);
215 }
216
217 void turnRight() {
218     analogWrite(ENA, velocidade);
219     analogWrite(ENB, velocidade);
220     digitalWrite(IN1, LOW);
221     digitalWrite(IN2, HIGH);
222     digitalWrite(IN3, HIGH);
223     digitalWrite(IN4, LOW);
224 }
225
226 void stopAll() {
227     digitalWrite(IN1, LOW);
228     digitalWrite(IN2, LOW);
229     digitalWrite(IN3, LOW);
230     digitalWrite(IN4, LOW);
231     analogWrite(ENA, 0);
232     analogWrite(ENB, 0);
233 }

```

Imagens do projeto:

